Lab 01

This lab has 2 different activities: one is hands-on (and was an example used in the lecture videos), one is computer-based. You can do them in any order.

1. Rocks (Module 1, Segment 4)

This activity is an opportunity for you to experiment with the Rocks example introduced in lecture. I recommend you go through the steps below and then watch again the video in Module 1 Segment 4 to see how your observations fit in with the whole class example in the lecture. Since the instructor knows the weights for each rock, the results were summarized in lecture.

Pictured above is a collection of 99 rocks. Your goal is to estimate the total weight of the 99 rocks. This can be done by estimating the average weight of a rock and multiplying by 99. The rocks are numbered, for your convenience.

1. Look at the rocks and choose the one that you believe best represents the average weight of the population. Record that rock number on your data sheet.
2. Look at the rocks and choose the five whose average you believe best represents the average weight of the population. Record those five rock numbers on your data sheet.
3. Look at the rocks and choose the five whose variability (think standard deviation) best represents the standard deviation of the population. Record those five rock numbers on your data sheet.

2. Introduction to Computing

Goals:

1. Learn more about R and RStudio
2. Learn how to read data into the format you want to use
3. Learn how to compute means, standard deviations, and draw histograms and box plots

Use of R and RStudio is demonstrated using the creativity data contained within the creativity.csv file.

Installing the Necessary Packages

If you haven't installed the `dplyr` and `ggplot2` packages yet, you can do so with the following calls to `install.packages()`.

```
install.packages('dplyr')    # only needs to be done once
install.packages('ggplot2')  # only needs to be done once.
```

You only need to do this once (repeat if you have installed a new version of R or are using R for the first time on a different computer). This downloads a bundle of functions, data, documentation, and sometimes compiled code from CRAN (or another repository), and stores it on your computer. After you install a package, it will be available to load into any future R session (unless you remove it). However, installing a package does not make its functions immediately usable. You will need to attach the package to your current R session to call them.

Loading the Necessary Packages

In order to take advantage of the functionality provided by each package, you'll need to attach it to your current session, which you can do with the following `library()` function calls.

```
# note, unlike install.packages(), the library names are not in quotes
library(dplyr)      # data manipulation library
library(ggplot2)    # GGplot graphics library
```

Reading and Summarizing Data

You can read in data from a comma delimited file with the `read.csv()` function. Notice, the first line contained in 'creativity.csv' is a header—a single row (usually the top row) that contains the names of each column. Therefore, we need to set the `header=TRUE` argument so the first row isn't mistakenly read in as a data observation.

```
# read in the data from the 'creativity.csv' file
# to reference 'creativity.csv' like this, it must exist within your working
# directory, see the R manual
creativity <- read.csv('creativity.csv', header=TRUE)
names(creativity)
```

```
[1] "treatment" "score"
```

This reads the data contained within `creativity.csv` as a data frame and stores it in a variable called `creativity`. The `names()` function returns the names (column names in this case) of this variable. You can print the first 6 observations with the `head()` function.

```
# change argument n to view more data (default n=6)
head(creativity)
```

```
  treatment score
1 intrinsic  12.0
2 intrinsic  12.0
3 intrinsic  12.9
4 intrinsic  13.6
5 intrinsic  16.6
6 intrinsic  17.2
```

You can also invoke the spreadsheet-style data viewer on the `creativity` data frame (or any matrix like object).

```
# will open a new tab in RStudio
View(creativity)
```

To get a quick six number summary of the numeric column score, you can call the `summary()` function, but you'll have to subset the data frame first with the `$` operator if you only want to return results for a particular column. Otherwise, you can omit the `$score` specification to summarize across all columns.

```
# call the summary function on the numeric column score
summary(creativity$score)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
   5.00   14.90   18.70   17.88   21.25   29.70
```

```
# for all variables
summarize(creativity)
```

```
data frame with 0 columns and 1 row
```

You can also use the 'pipe' operator `|>` to achieve the same goals. `|>` takes the result of one expression and passes it as the first argument to the next function. Popular in tidyverse-style programming, this makes code easier to read, write, and understand by expressing operations as a clear sequence of steps.

```
# print first 6 observations
# white space doesn't matter; it's good practice to use newlines and
# indentation to make your code easier to read
creativity |>
    head()
```

```
  treatment score
1 intrinsic  12.0
2 intrinsic  12.0
3 intrinsic  12.9
4 intrinsic  13.6
5 intrinsic  16.6
6 intrinsic  17.2
```

```
# the select() function allows users to select and (optionally rename)
# columns in a data frame
creativity |>
  select(score) |>
  head()
```

```
  score
1  12.0
2  12.0
3  12.9
4  13.6
5  16.6
6  17.2
```

```
# return six number summary of only the score column
creativity |>
  select(score) |>
  summary()
```

```
     score
 Min.   : 5.00
 1st Qu.:14.90
 Median :18.70
 Mean   :17.88
 3rd Qu.:21.25
 Max.   :29.70
```

```
# return the mean of the score column
creativity |>
  pull(score) |>
  mean()
```

```
[1] 17.8766
```

To return summary calculations as a data frame, you can call the `summarize()` function. Specify the name of the calculation column (here it's 'meanScore') and then execute the proper expression.

```
# calculate the mean Score, return as data frame with column meanScore
creativity |>
  summarize(
      meanScore=mean(score)
      )
```

```
  meanScore
1   17.8766
```

You can also specify multiple summarizing expressions

```
# calculate the mean and median of the score column
creativity |>
  summarize(
    meanScore = mean(score),
    medianScore = median(score)
    )
```

```
  meanScore medianScore
1   17.8766        18.7
```

You can perform calculations by group, just specify the grouping column in a `group_by()` statement.

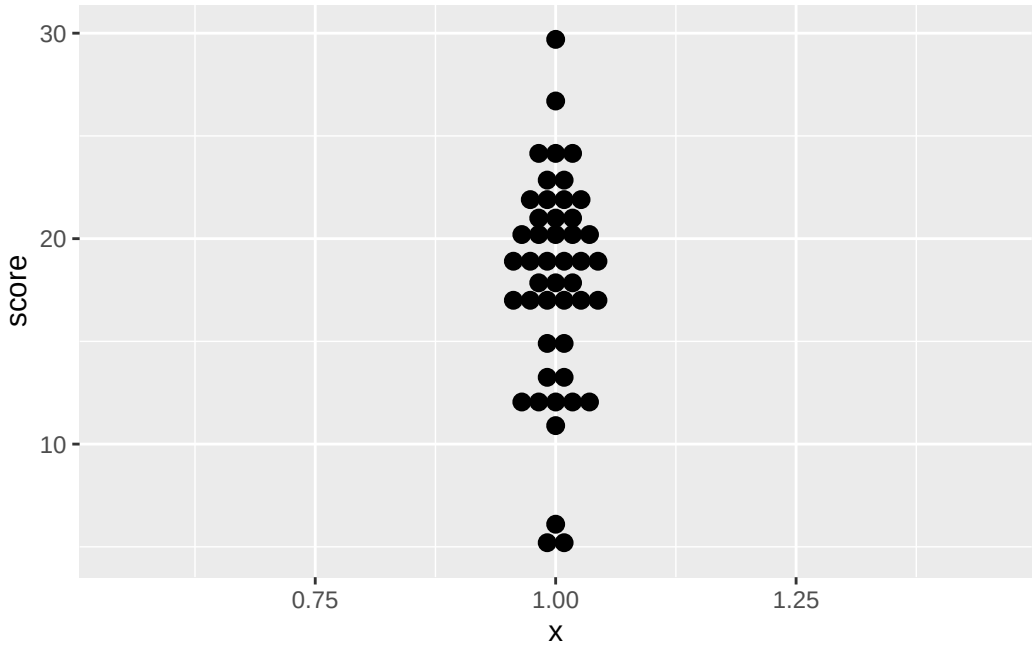
```
# calculate the mean and median score by treatment group
creativity |>
  group_by(treatment) |>
   summarize(
    meanScore = mean(score),
    medianScore = median(score)
    )
```

```
# A tibble: 2 x 3
  treatment meanScore medianScore
  <chr>         <dbl>       <dbl>
1 extrinsic      15.8        17.2
2 intrinsic      19.9        20.4
```

Plotting Data

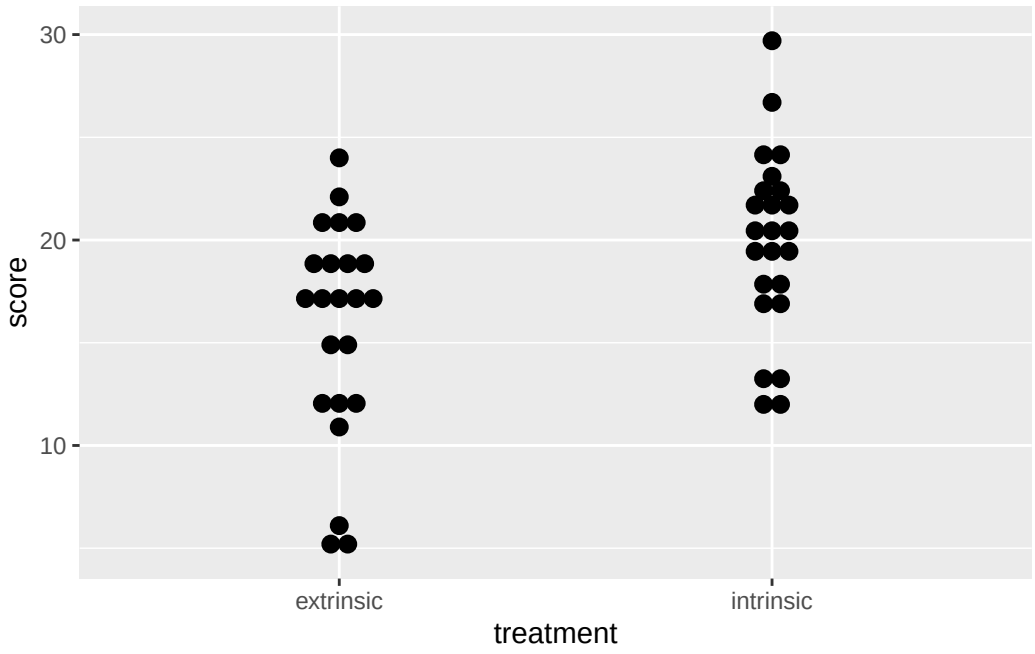
To build a dotplot of the data, call `ggplot()`. In the `ggplot2` package, functions usually require a data frame as the first argument, along with aesthetic mappings. Aesthetics (modified by the `aes` argument) map variables to visual properties of the graph, like the x-axis and y-axis. 'geom' layers—usually functions that start with `geom_*()` define which graphics to draw. The `+` operator combines all layers into a single composition.

```
# create dot plot
# specify x as 1, all dots will be stacked centered at the vertical line x=1
# y is the score variable
# binaxis is the axis to bin along, here y which is score
# stackdir specifies which direction to stack the dots
ggplot(creativity, aes(x=1, y=score)) +
    geom_dotplot(binaxis='y', stackdir='center')
```



You can also create dotplots for groups of data

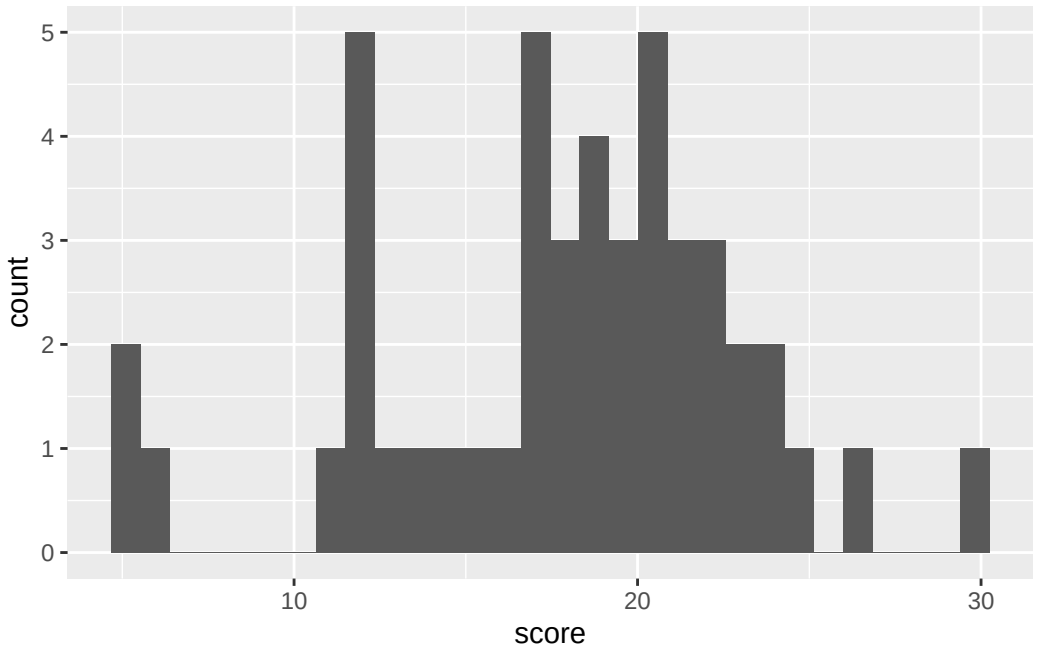
```
# side-by-side dotplots of two (or more) groups
ggplot(creativity, aes(x=treatment, y = score)) +
    geom_dotplot(binaxis='y', stackdir='center')
```

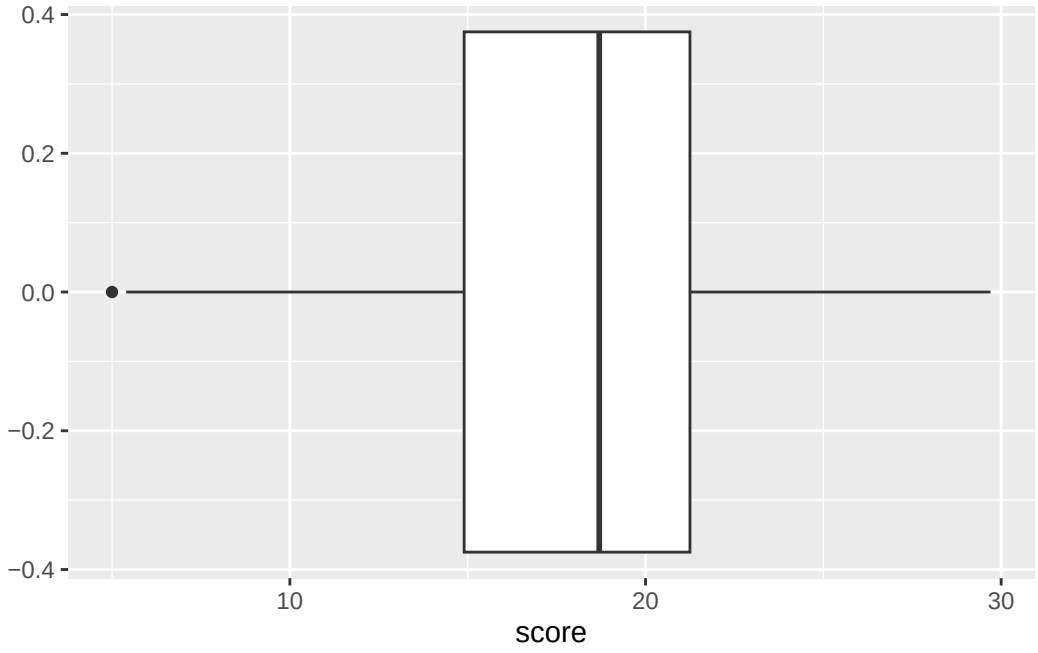


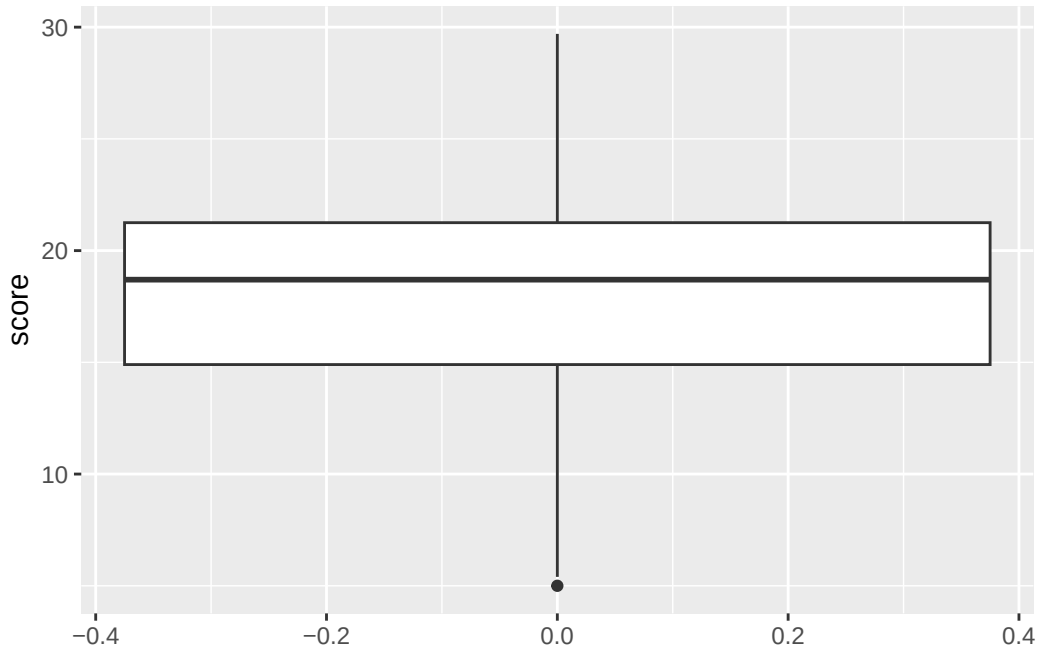
To avoid redundancy and repeated calls to `ggplot()`, it's usually best practice to build a template plot and then call the geom layers downstream. This separates the setup from the actual plotting.

```
# template plot, no graphics drawn
creativity_setup1 <- ggplot(creativity, aes(x=score))

# call geom functions separately
creativity_setup1 +
    geom_histogram()
creativity_setup1 +
    geom_boxplot()
creativity_setup1 +
    coord_flip() +
    geom_boxplot()
```


count
5
4
3
2
1
0
10
20
30
score


0.4
0.2
0.0
−0.2
−0.4
10
20
30
score

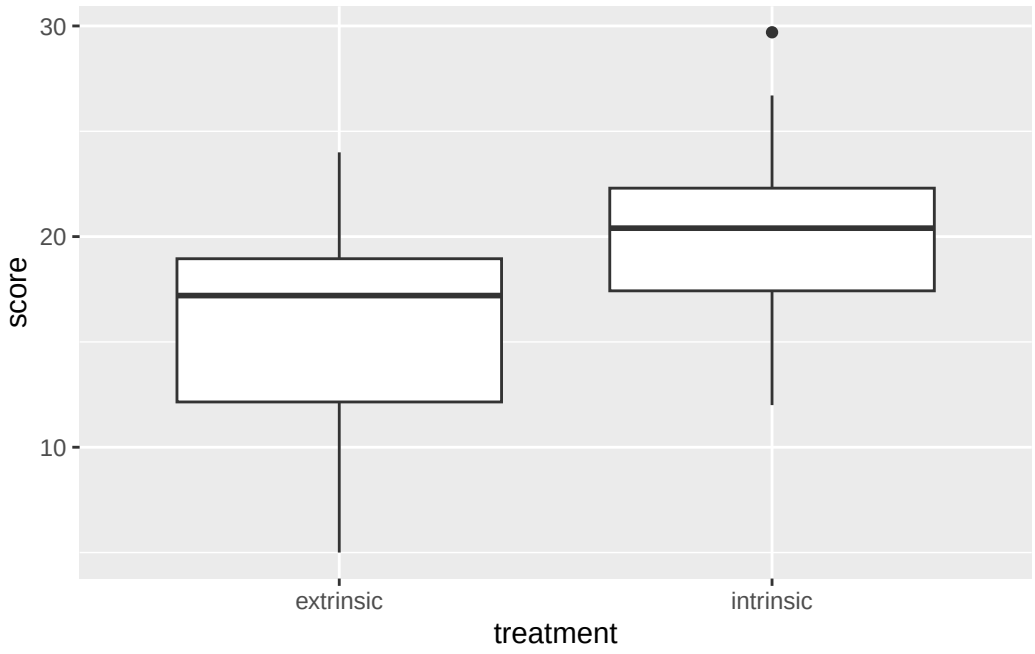


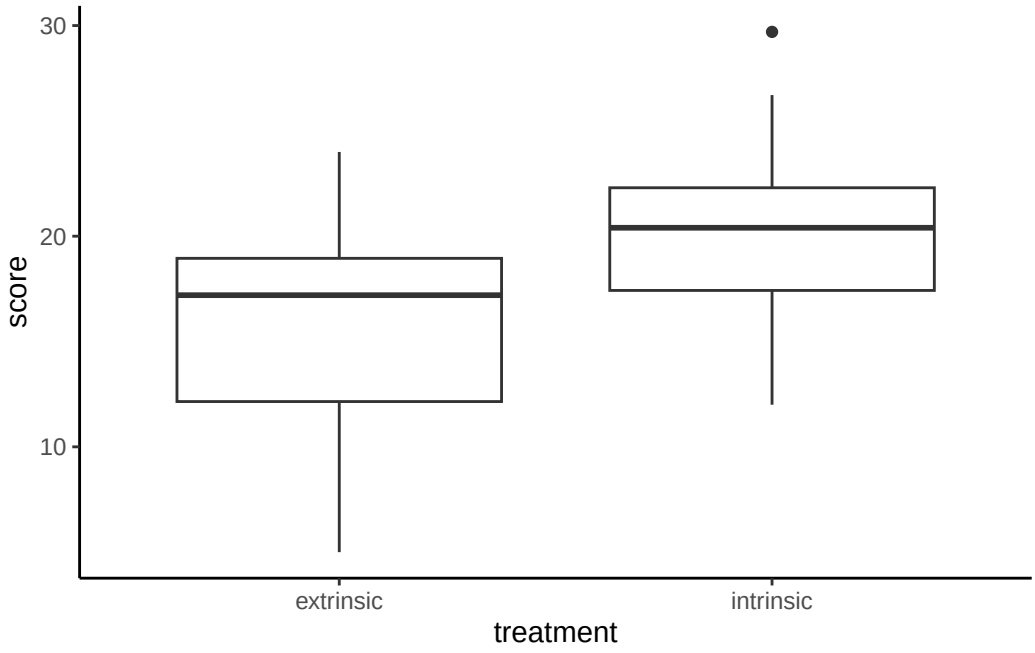
This next example changes the theme to classic and plots for multiple groups

```
# new template that groups by treatment
creativity_setup2 <- ggplot(creativity, aes(x=treatment, y=score))

# draw box plot
creativity_setup2 +
    geom_boxplot()

# change theme and draw the same box plot
creativity_setup2 +
    theme_classic() +
    geom_boxplot()
```





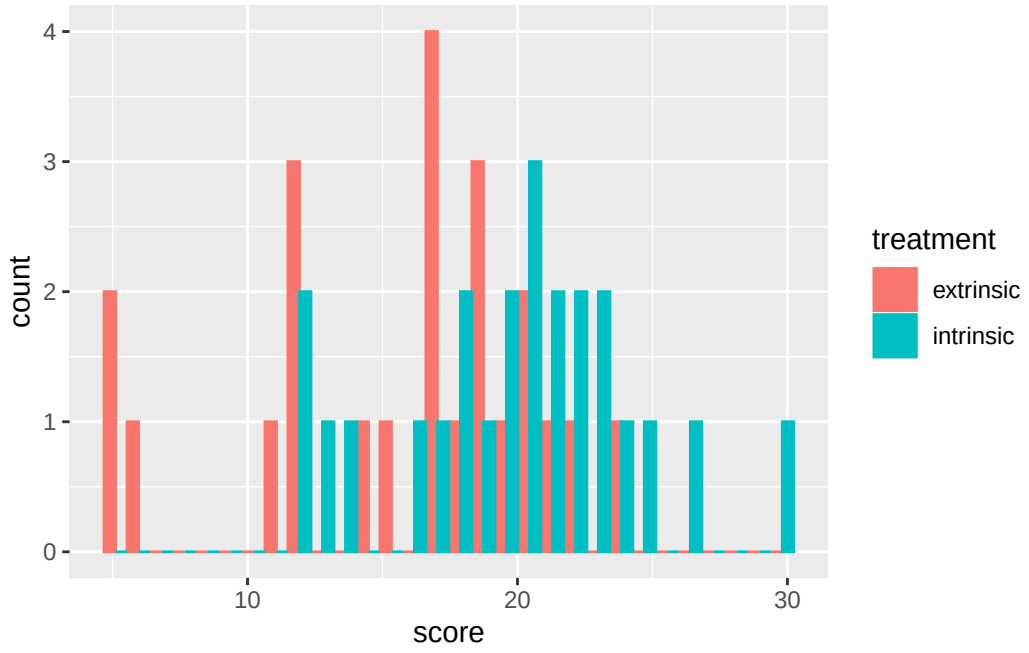
You can also specify color with the aesthetic mappings argument.

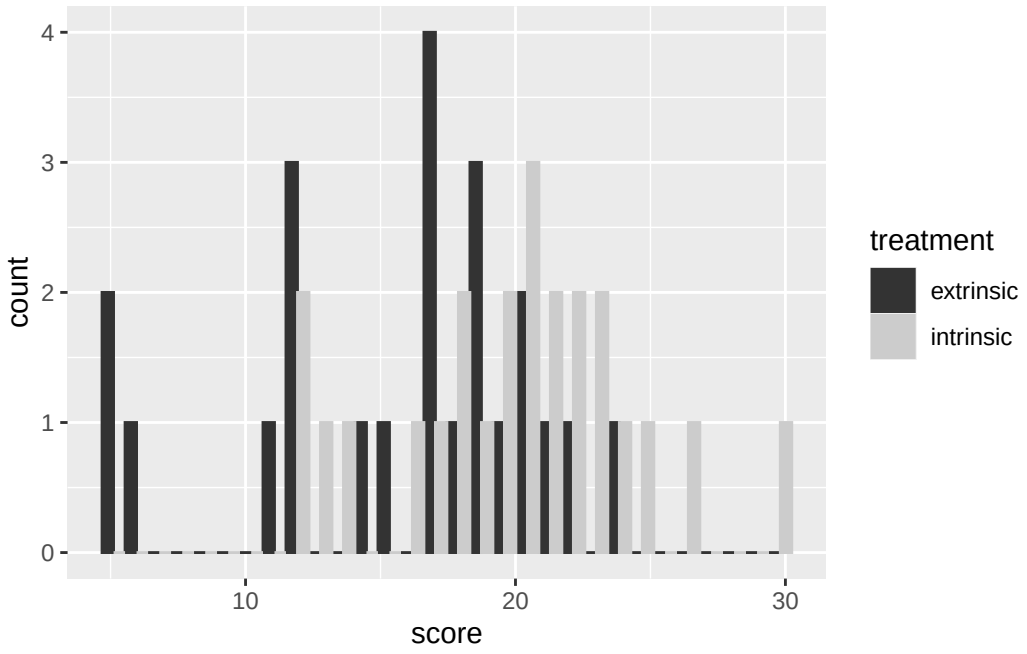
```
# add color to visually separate groups
# template plot, no graphics drawn
```

```r
creativity_setup3 <- ggplot(creativity, aes(x=score,
                                            fill=treatment,
                                            color=treatment))

# draw histogram
creativity_setup3 +
    geom_histogram(position='dodge')

# draw histogram, change color scale
creativity_setup3 +
    scale_color_grey() +
    scale_fill_grey() +
    geom_histogram(position='dodge')
```





You can save the last displayed plot directly in your working directory with the `ggsave()` function. You can set the plot size, define the file name and path, and specify the device function (png, jpeg, tiff, etc).

```
# by default, this will save in your working directory
ggsave('histogram.png')
```

Details and Context

Comments

The # symbol starts a comment. All text from the # to the next end-of-line is ignored.

Installing and Loading Packages

R is open-source software with a huge number (currently almost 20,000) of add-on packages contributed by users. We will use a variety of these in this course. The code presented here will call functions in two of these packages. There are two steps to be able to use an add-on packages:

```
install.packages('dplyr')
```

This copies the package from a repository to your computer. This only needs to be done once (at least until you install a new version of R). Note that the package name is in quotes here.

```
library(dplyr)
```

This links that package to your current R session, and it needs to be executed every time you start an R session. Functions in the add-on library will only be available if you link that library. Note that the library name **is not in quotes** here.

Why do you have to link a library every time? Sometimes an R library will include functions with the same name as functions in another library (or even in base R). For example, the function `union()` is defined in base R and again in `dplyr`. Using all libraries at the same time would be chaos (which version of `union()` did you really want to run?). By default, when there are name conflicts, R will use the function in the last defined library. For example, when you run `library(dplyr)` you (should) see a message that the states certain functions of the same name were masked by the latest package attachment.

Working Directory

```
getwd()
```

This prints the default working directory. If your desired working directory is a folder under this, you can use a relative path in the `setwd()` function.

```
# can be a relative path or an absolute path
setwd('path to folder')
```

This sets the working directory. Can use a relative path or an absolute path, i.e., starting at `C:` or some other drive. R will look for code and data or save files in the specified directory.

I strongly recommend creating a folder for all your course work. Then create a separate folder for each project you work on.

Piping

Lines of R code can often become hard to read. Piping is a way to organize code so it's easier to read. The native piping operator—as of R version 4.1.0—is `|>` (a pipe followed by a greater-than character). If you're familiar with piping in other computer languages and operating systems, this exactly the same. The pipe evaluates the left-hand side. The result of that is passed as the first argument to the function on the right-hand side. (I'm omitting details about order of arguments because they usually don't matter).

```
# piping to head()
creativity |> head()
```

The creativity data frame is supplied as the first argument of the `head()` function which displays the contents in interactive settings (like the R console and RStudio) if this isn't part of an assignment statement. This does the same thing as

```
# explicit call to head()
head(creativity)
```

Important Note: If you break the command into multiple lines, it is easier to read. Notice that the `|>` is at the end of the line. That is crucial. If you tried to run:

```
# these two lines will result in error
creativity
|> head()
```

you'll get a printout of everything in the creativity data set followed by an error that is hard to interpret. What's going on? Remember that a set of R commands can continue on multiple lines only when R expects more to come. In the bad version, creativity is a stand-alone command, but:

```
# this tells R to expect more on the from the next line
creativity |>
```

is not—R expects something on the right-hand side of the pipe. You"ll see a `+` when the pair of commands is echoed on the console. Note: if breaking commands onto multiple lines is confusing, you can always put everything on a single line. **However** this can be much harder to read. Remember you may need to look at your own code again 6 or 12 months after you wrote it.

Wrangling

The `select()` function from `dplyr` selects the specified variables in a data frame. It takes a data frame, e.g. piped into it, and returns a new data frame with only the specified variables. So:

```
creativity |> select(score)
```

returns a data frame with only the specified variables. **Advanced Uses:** `select()` allows all sorts of special ways to specify variables, not just by name. See texts or online resources on data management in R for details. See if you can predict what the following does before running the code:

```
creativity |>
    select(score) |>
    head()
```

Similarly, the `pull()` function is used to extract a single column from a data frame (or tibble) as a vector. Unlike `select()`, `pull()` does not return a data frame. It returns a vector of the single, specified column.

```
creativity |>
    pull(score)
```

Grouping and Summarizing

`summarize()` takes multiple observations and returns a summary statistic, e.g., a mean or a median. Each summary is requested by strings of new variable name: `new_variable_name = function(old_variable_name)`. So

```
creativity |>
    summarize(meanScore = mean(score))
```

will create a new data frame with one new variable `meanScore` that is the average of all the values of score. You can create summaries of multiple variables or multiple summaries of the same variable by adding additional requests, separated by commas inside the `summarize()` expression. Look carefully at where the parentheses are.

The creativity data set has data on 2 treatments. You probably want to see the mean for each treatment. `dplyr` functions make this easy and return a very interpretable data set. `group_by()` Adds grouping information to a data frame. Subsequent operations are done on

each group of observations, not on all observations. To get the mean and median score for each treatment, just insert `group_by()` in the chain of pipes between the data frame name and the `summarize()` operations

```
creativity |>
    group_by(treatment) |>
    summarize(meanScore = mean(score))
```

Plotting

`ggplot2` is an add-on library that provides extremely powerful ways to produce graphs. Like most powerful systems, it can be a bit intimidating to get started. We'll start with simple things. We use `ggplot2` and `ggplot` interchangeably. The language is called `ggplot` and was first implemented in the `ggplot` library. `ggplot2` is an updated library.

Basic Structure of ggplot Commands: Three parts—a data frame name, an aesthetic, and one or more plot command. The first two are arguments to the `ggplot()` function. The last is done by functions that start with `geom_*`. The output from the `ggplot()` function is passed to the plot function. The two parts are separated by a `+`.

Aesthetics: These are specified by the `aes()` function. The minimum contents are the names of the x and y variables. The `aes()` function is also where you specify colors and other options. Examples are below

Advanced Note: If this sounds like piping, it is. The `+` does the same thing as the pipe operator `|>`. `ggplot()` was written before piping was written. Folks are revising ggplot to use pipes, but that is in the future.

Separating Setup from Plotting: if you use the *data* `+` *aesthetic* structure for multiple plots, you can build a template and store the results in a variable. `creativity_setup1` saves the result for the creativity data set with the score as the x-axis. An x variable is required (even if ignored), but the y variable doesn't need to specified if not needed. This saved setup can be used for multiple plots. For example:

```
# template plot, no graphics drawn
creativity_setup1 <- ggplot(creativity, aes(x=score))

# histogram
# draws a histogram with scores on the x axis and frequency on the y
creativity_setup1 +
    geom_histogram()

# horizontal boxplot
# draws a horizontal box plot (because score is on the x axis
```

```
creativity_setup1 +
    geom_boxplot()

# boxplot
# flips the x and y axis (so score is now on the y axis) and draws a box plot
# this is probably the box plot you want to draw
creativity_setup1 +
    coord_flip() +
    geom_boxplot()
```

To create a vertical dotplot of one group of data we can use the `geom_dotplot()` function; we won't create and store a setup template for this though. Instead, the setup `ggplot()` function specifies the data frame: creativity. Since this code produces a vertical dotplot, the `aes()` function names a y variable. Since an x variable is required but not used for a single plot, we declare `x=1`. `geom_dotplot()` draws a dot plot. The best-looking ones produced by binning values and centering them in the plot. Those features are specified by `binaxis="y"`, `stackdir="center"`. You're welcome to play with these arguments, but I recommend you just do them as given. If you want to know more or look at alternatives, look in the documentation. **Important:** the `+`. Notice the ggplot() command ends on the 3rd line of the common before the `+`. The `+` tells R that the command is incomplete and to expect more. The plotting command is on the next line.

```
# if y is continuous and you want to bin along this axis, set binaxis="y"
ggplot(creativity, aes(x=1, y=score)) +
    geom_dotplot(binaxis="y", stackdir="center")
```

To draw side-by-side plots for the two treatments, declare the x variable as `x=treatment` in the aesthetic mappings. We want treatment on the x-axis and score on the y-axis. Once you've defined that aesthetic, the actual plotting is identical to before.

```
# same as before, but x=treatment now
ggplot(creativity, aes(x=treatment, y=score)) +
    geom_dotplot(binaxis="y", stackdir="center")

# multiple groups for histograms
# it's usually clearest if the two groups are drawn side-by-side instead of
# overplotting them
# you can change this with the position='dodge' argument of the
# geom_histogram() function
ggplot(creativity, aes(x=score, fill=treatment, color=treatment)) +
    geom_histogram(position="dodge")
```

Cleaning up the Plot: the default ggplot graphic uses a grey background with white marker lines. Graphs for publications usually don't have these. The ggplot system includes multiple themes that control overall features of a graph. `theme_classic()` is the theme for graphics without bells and whistles. Insert that between the setup and the plotting function.

```
creativity_setup1 +
    theme_classic() +
    geom_histogram()
```

You can also use grey shades instead of colors:

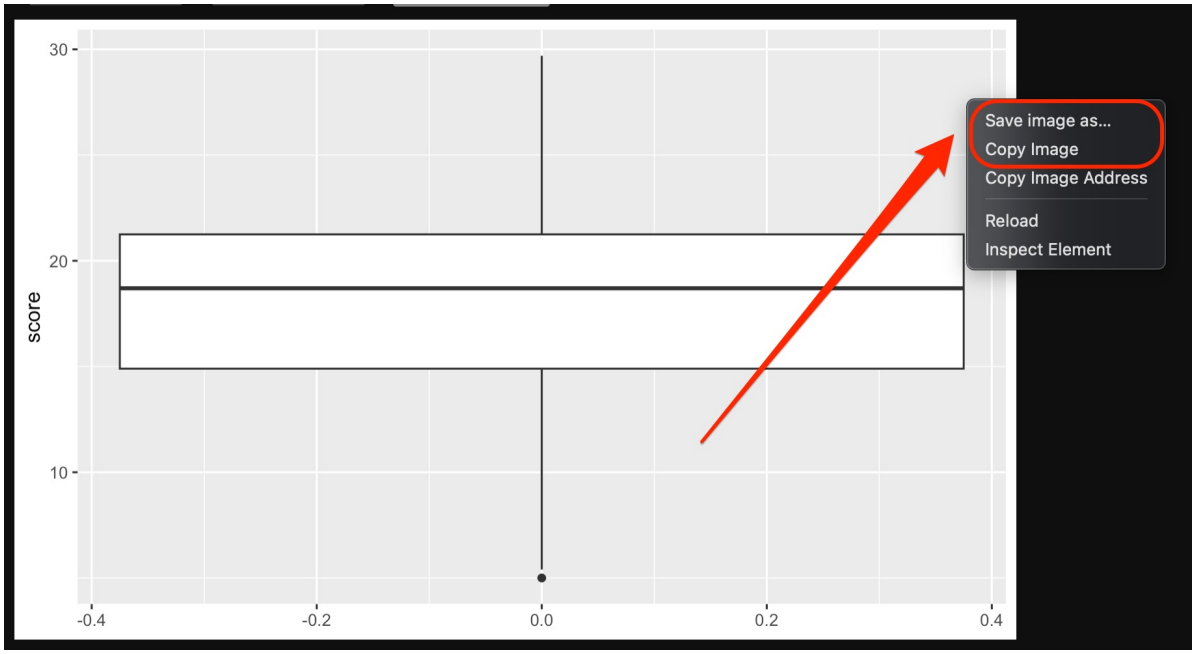
```
creativity_setup1 +
    theme_classic() +
    geom_histogram() +
    scale_color_grey() +
    scale_fill_grey()
```

Saving Figures

For the last displayed plot, you can call `ggsave()` to save the figure as a file. Specify an absolute path to save anywhere or a relative path to save within your working directory

```
ggsave("figure.png")
```

In RStudio, you can also right click on the figure and select Copy Image or Save image as…



Self Assessment

Exercise

Read in the tomato data, `tomato.csv`. These are the values used last week in the "paper" exercise. This is a randomized experiment comparing a proposed "better" fertilizer to my usual fertilizer. Seven plants (treatment a) were randomly assigned to the "better" fertilizer; the other seven plants received the usual fertilizer. The measurement units are pounds of ripe tomato.

1. draw side-by-side box plots of the yield for each of the two fertilizers
2. estimate the difference in mean yield between the two fertilizers (a-b). Include units.
3. estimate the difference in median yield between the two fertilizers (a-b). include units.

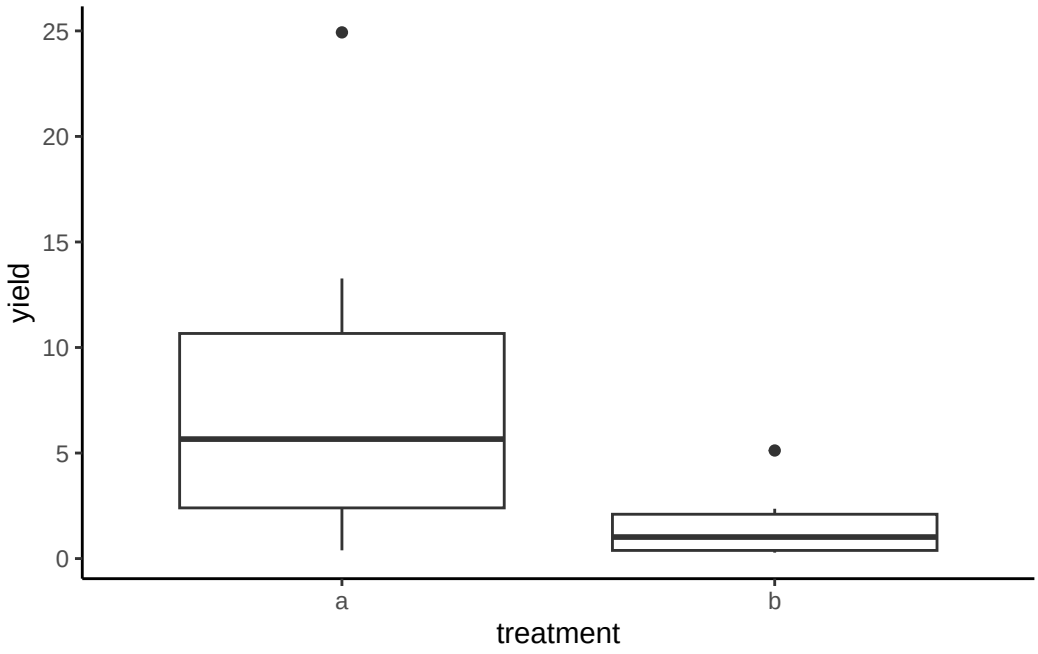
Solution

1.

```r
# read in the data
tomato <- read.csv("tomato.csv", header=TRUE)
head(tomato)

# draw side by side boxplots of the yield by treatment
ggplot(tomato) +
    geom_boxplot(aes(x=treatment, y=yield)) +
    theme_classic()
```

```
  treatment yield
1         a  0.39
2         b  0.44
3         a  1.83
4         a  5.66
5         b  0.33
6         a  8.06
```



2.

```r
# estimate the difference in mean yield between the two fertilizers
tomato |>
```

```
    summarize(diff=yield[treatment=="a"]-yield[treatment=="b"]) |>
    pull(diff) |>
    mean()
```

```
[1] 6.531429
```

or alternatively:

```
tomato |>
    group_by(treatment) |>
    summarize(mean_yield=mean(yield)) |>
    summarize(diff=diff(rev(mean_yield))) # rev forces correct diff order
```

```
# A tibble: 1 x 1
   diff
  <dbl>
1  6.53
```

3.

```
# estimate the difference in median yield between the two fertilizers
tomato |>
    summarize(diff=yield[treatment=="a"]-yield[treatment=="b"]) |>
    pull(diff) |>
    median()
```

```
[1] 4.64
```